

```
// ===== 文件: src/main.js =====
import 'element-ui/lib/theme-chalk/index.css';
import 'normalize.css/normalize.css'; // A modern alternative to CSS resets
import Vue from 'vue';
import ElementUI from 'element-ui';
import App from './App.vue';
import router from './router';
import store from './store';
import i18n from './i18n';
import locale from 'element-ui/lib/locale'
import './styles/global.scss';
import { register as registerServiceWorker } from './registerServiceWorker';
import featureManager from './utils/featureManager';

// 创建事件总线。用于组件间通信
Vue.prototype.$eventBus = new Vue();

Vue.use(ElementUI);
locale.i18n((key, value) => i18n.t(key, value))

Vue.config.productionTip = false

// 注册Service Worker
registerServiceWorker();

// 创建Vue实例
new Vue({
  router,
  store,
  i18n,
  render: function (h) { return h(App) }
}).$mount('#app')
// ===== 文件: src/App.vue =====
<template>
  <div id="app">
    <router-view />
    <cache-viewer v-if="isCDNEnabled" :visible.sync="showCacheViewer" />
  </div>
</template>

<style lang="scss">
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
}

nav {
  padding: 30px;

  a {
    font-weight: bold;
    color: #2c3e50;

    &.router-link-exact-active {
      color: #42b983;
    }
  }
}
```

```
}

.copyright {
  padding: 0 !important;
  color: rgb(0, 0, 0);
  font-size: 12px;
  font-weight: 400;
  margin-top: auto;
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
}

.el-message {
  top: 70px !important;
}
</style>
<script>
import CacheViewer from '@components/CacheViewer.vue';
import { logCacheStatus } from '@utils/cacheViewer';

export default {
  name: 'App',
  components: {
    CacheViewer
  },
  data() {
    return {
      showCacheViewer: false,
      isCDNEnabled: process.env.VUE_APP_USE_CDN === 'true'
    };
  },
  created() {
    // 挂载 store 状态
    this.$store.commit('setUserInfo', JSON.parse(localStorage.getItem('userInfo') || '{}'));
    this.$store.commit('setPubConfig', JSON.parse(localStorage.getItem('pubConfig') || '{}'));
  },
  mounted() {
    // 检测是否为移动设备且VUE_APP_H5_URL不为空，如果两个条件都满足则跳转到H5页面
    if (this.isMobileDevice() && process.env.VUE_APP_H5_URL) {
      window.location.href = process.env.VUE_APP_H5_URL;
      return;
    }

    // 只有在启用CDN时才添加相关事件和功能
    if (this.isCDNEnabled) {
      // 添加全局快捷键Alt+C用于显示缓存查看器
      document.addEventListener('keydown', this.handleKeyDown);

      // 在全局对象上添加缓存检查方法，便于调试
      window.checkCDNCacheStatus = () => {
        this.showCacheViewer = true;
      };

      // 在控制台输出提示信息
      console.info(
        '%c[' + this.$t('system.name') + ']' + ' ' + this.$t('cache.cdnEnabled'),
        'color: #409EFF; font-weight: bold;'
      );
    }
  }
}
```

```
);
console.info(
  '按下 Alt+C 组合键或在控制台运行 checkCDNCacheStatus() 可以查看CDN缓存状态'
);

// 检查Service Worker状态
this.checkServiceWorkerStatus();
} else {
  console.info(
    '%c[' + this.$t('system.name') + '] ' + this.$t('cache.cdnDisabled'),
    'color: #67C23A; font-weight: bold;'
  );
}
},
beforeDestroy() {
  // 只有在启用CDN时才需要移除事件监听
  if (this.isCDNEnabled) {
    document.removeEventListener('keydown', this.handleKeyDown);
  }
},
methods: {
  handleKeyDown(e) {
    // Alt+C 快捷键
    if (e.altKey && e.key === 'c') {
      this.showCacheViewer = true;
    }
  },
  isMobileDevice() {
    // 检测是否为移动设备的函数
    return /Android|webOS|iPhone|iPad|iPod|BlackBerry|IEMobile|Opera Mini/i.test(navigator.userAgent);
  },

  async checkServiceWorkerStatus() {
    // 检查Service Worker是否已注册
    if ('serviceWorker' in navigator) {
      try {
        const registrations = await navigator.serviceWorker.getRegistrations();
        if (registrations.length > 0) {
          console.info(
            '%c[' + this.$t('system.name') + '] ' + this.$t('cache.serviceWorkerRegistered'),
            'color: #67C23A; font-weight: bold;'
          );
        }

        // 输出缓存状态到控制台
        setTimeout(async () => {
          const hasCaches = await logCacheStatus();
          if (!hasCaches) {
            console.info(
              '%c[' + this.$t('system.name') + '] ' + this.$t('cache.noCacheDetected'),
              'color: #E6A23C; font-weight: bold;'
            );
          }

          // 开发环境下提供额外提示
          if (process.env.NODE_ENV === 'development') {
            console.info(
              '%c[' + this.$t('system.name') + '] ' + this.$t('cache.swDevEnvWarning'),
              'color: #E6A23C; font-weight: bold;'
            );
            console.info(this.$t('cache.swCheckMethods'));
            console.info('1. ' + this.$t('cache.swCheckMethod1'));
          }
        }, 1000);
      } catch (error) {
        console.error('Service Worker 检查失败: ', error);
      }
    }
  }
}
```

```

        console.info('2. ' + this.$t('cache.swCheckMethod2'));
        console.info('3. ' + this.$t('cache.swCheckMethod3'));
    }
}
}, 2000);
} else {
    console.info(
        '%c[' + this.$t('system.name') + '] ' + this.$t('cache.serviceWorkerNotRegistered'),
        'color: #F56C6C; font-weight: bold;'
    );

    if (process.env.NODE_ENV === 'development') {
        console.info(
            '%c[' + this.$t('system.name') + '] ' + this.$t('cache.swDevEnvNormal'),
            'color: #E6A23C; font-weight: bold;'
        );
        console.info(this.$t('cache.swProdOnly'));
        console.info(this.$t('cache.swTestingTitle'));
        console.info('1. ' + this.$t('cache.swTestingStep1'));
        console.info('2. ' + this.$t('cache.swTestingStep2'));
    }
}
} catch (error) {
    console.error('检查Service Worker状态失败:', error);
}
} else {
    console.warn(this.$t('cache.swNotSupported'));
}
}
}
};
</script>
// ===== 文件: src/registerServiceWorker.js =====
/* eslint-disable no-console */

export const register = () => {
    if (process.env.NODE_ENV === 'production' && 'serviceWorker' in navigator) {
        window.addEventListener('load', () => {
            const swUrl = `${process.env.BASE_URL}service-worker.js`;

            console.info(`[小智服务] 正在尝试注册Service Worker, URL: ${swUrl}`);

            // 先检查Service Worker是否已注册
            navigator.serviceWorker.getRegistrations().then(registrations => {
                if (registrations.length > 0) {
                    console.info('[小智服务] 发现已有Service Worker注册, 正在检查更新');
                }

                // 继续注册Service Worker
                navigator.serviceWorker
                    .register(swUrl)
                    .then(registration => {
                        console.info('[小智服务] Service Worker注册成功');

                        // 更新处理
                        registration.onupdatefound = () => {
                            const installingWorker = registration.installing;
                            if (installingWorker == null) {
                                return;
                            }
                        }
                    }
                );
            }
        });
    }
};

```

```

installingWorker.onstatechange = () => {
  if (installingWorker.state === 'installed') {
    if (navigator.serviceWorker.controller) {
      // 内容已缓存更新, 通知用户刷新
      console.log('[小智服务] 新内容可用, 请刷新页面');
      // 可以在这里展示更新提示
      const updateNotification = document.createElement('div');
      updateNotification.style.cssText = `
        position: fixed;
        bottom: 20px;
        right: 20px;
        background: #409EFF;
        color: white;
        padding: 12px 20px;
        border-radius: 4px;
        box-shadow: 0 2px 12px 0 rgba(0,0,0,.1);
        z-index: 9999;
      `;
      updateNotification.innerHTML = `
        <div style="display: flex; align-items: center;">
          <span style="margin-right: 10px;">发现新版本, 点击刷新应用</span>
          <button style="background: white; color: #409EFF; border: none; padding: 5px 10px; border-radius: 3px; cursor: pointer;">刷新</button>
        </div>
      `;
      document.body.appendChild(updateNotification);
      updateNotification.querySelector('button').addEventListener('click', () => {
        window.location.reload();
      });
    } else {
      // 一切正常, Service Worker已成功安装
      console.log('[小智服务] 内容已缓存供离线使用');

      // 可以在这里初始化缓存
      setTimeout(() => {
        // 预热CDN缓存
        const cdnUrls = [
          'https://unpkg.com/element-ui@2.15.14/lib/theme-chalk/index.css',
          'https://cdnjs.cloudflare.com/ajax/libs/normalize/8.0.1/normalize.min.css',
          'https://unpkg.com/vue@2.6.14/dist/vue.min.js',
          'https://unpkg.com/vue-router@3.6.5/dist/vue-router.min.js',
          'https://unpkg.com/vuex@3.6.2/dist/vuex.min.js',
          'https://unpkg.com/element-ui@2.15.14/lib/index.js',
          'https://unpkg.com/axios@0.27.2/dist/axios.min.js',
          'https://unpkg.com/opus-decoder@0.7.7/dist/opus-decoder.min.js'
        ];

        // 预热缓存
        cdnUrls.forEach(url => {
          fetch(url, { mode: 'no-cors' }).catch(err => {
            console.log(`预热缓存 ${url} 失败`, err);
          });
        });
      }, 2000);
    }
  }
};

```

```

        console.error('Service Worker 注册失败:', error);

        if (error.name === 'TypeError' && error.message.includes('Failed to register a ServiceWorker
    ')) {
            console.warn('[小智服务] 注册Service Worker时出现网络错误, CDN资源可能无法缓存');
            if (process.env.NODE_ENV === 'production') {
                console.info(
                    '可能原因: 1. 服务器未配置正确的MIME类型 2. 服务器SSL证书问题 3. 服务器未返回service-w
orker.js文件'
                );
            }
        }
    });
});
});
};

export const unregister = () => {
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.ready
            .then(registration => {
                registration.unregister();
            })
            .catch(error => {
                console.error(error.message);
            });
    }
};

// ===== 文件: src/service-worker.js =====
/* global self, workbox */

// 自定义Service Worker安装和激活的处理逻辑
self.addEventListener('message', (event) => {
    if (event.data && event.data.type === 'SKIP_WAITING') {
        self.skipWaiting();
    }
});

// CDN资源列表
const CDN_CSS = [
    'https://unpkg.com/element-ui@2.15.14/lib/theme-chalk/index.css',
    'https://cdnjs.cloudflare.com/ajax/libs/normalize/8.0.1/normalize.min.css'
];

const CDN_JS = [
    'https://unpkg.com/vue@2.6.14/dist/vue.min.js',
    'https://unpkg.com/vue-router@3.6.5/dist/vue-router.min.js',
    'https://unpkg.com/vuex@3.6.2/dist/vuex.min.js',
    'https://unpkg.com/element-ui@2.15.14/lib/index.js',
    'https://unpkg.com/axios@0.27.2/dist/axios.min.js',
    'https://unpkg.com/opus-decoder@0.7.7/dist/opus-decoder.min.js'
];

// 当Service Worker被注入manifest后会自动执行
const manifest = self.__WB_MANIFEST || [];

// 检查是否启用CDN模式
const isCDNEnabled = manifest.some(entry =>
    entry.url === 'cdn-mode' && entry.revision === 'enabled'

```

```
);

console.log(`Service Worker 已初始化, CDN模式: ${isCDNEnabled ? '启用' : '禁用'}`);

// 注入workbox相关代码
importScripts('https://storage.googleapis.com/workbox-cdn/releases/7.0.0/workbox-sw.js');
workbox.setConfig({ debug: false });

// 开启workbox
workbox.core.skipWaiting();
workbox.core.clientsClaim();

// 预缓存离线页面
const OFFLINE_URL = '/offline.html';
workbox.precaching.precacheAndRoute([
  { url: OFFLINE_URL, revision: null }
]);

// 添加安装完成事件处理器, 在控制台显示安装消息
self.addEventListener('install', event => {
  if (isCDNEnabled) {
    console.log('Service Worker 已安装, 开始缓存CDN资源');
  } else {
    console.log('Service Worker 已安装, CDN模式禁用, 仅缓存本地资源');
  }
});

// 确保离线页面被缓存
event.waitUntil(
  caches.open('offline-cache').then((cache) => {
    return cache.add(OFFLINE_URL);
  })
);

// 添加激活事件处理器
self.addEventListener('activate', event => {
  console.log('Service Worker 已激活, 现在控制着页面');

  // 清理旧版本缓存
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.filter(cacheName => {
          // 清理除当前版本外的缓存
          return cacheName.startsWith('workbox-') && !workbox.core.cacheNames.runtime.includes(cacheName)
        })
      ).map(cacheName => {
        return caches.delete(cacheName);
      })
    )
  );
});

// 添加fetch事件拦截器, 用于查看CDN资源是否命中缓存
self.addEventListener('fetch', event => {
  // 只有启用CDN模式时才进行CDN资源缓存监控
  if (isCDNEnabled) {
    const url = new URL(event.request.url);
```

```
// 针对CDN资源，输出是否命中缓存的信息
if ([...CDN_CSS, ...CDN_JS].includes(url.href)) {
  // 不干扰正常的fetch流程，只添加日志
  console.log(`请求CDN资源: ${url.href}`);
}
});

// 仅在CDN模式下缓存CDN资源
if (isCDNEnabled) {
  // 缓存CDN的CSS资源
  workbox.routing.registerRoute(
    ({ url }) => CDN_CSS.includes(url.href),
    new workbox.strategies.CacheFirst({
      cacheName: 'cdn-stylesheets',
      plugins: [
        new workbox.expiration.ExpirationPlugin({
          maxAgeSeconds: 365 * 24 * 60 * 60, // 增加到1年缓存
          maxEntries: 10, // 最多缓存10个CSS文件
        }),
        new workbox.cacheableResponse.CacheableResponsePlugin({
          statuses: [0, 200], // 缓存成功响应
        }),
      ],
    })
  );

  // 缓存CDN的JS资源
  workbox.routing.registerRoute(
    ({ url }) => CDN_JS.includes(url.href),
    new workbox.strategies.CacheFirst({
      cacheName: 'cdn-scripts',
      plugins: [
        new workbox.expiration.ExpirationPlugin({
          maxAgeSeconds: 365 * 24 * 60 * 60, // 增加到1年缓存
          maxEntries: 20, // 最多缓存20个JS文件
        }),
        new workbox.cacheableResponse.CacheableResponsePlugin({
          statuses: [0, 200], // 缓存成功响应
        }),
      ],
    })
  );
}

// 无论是否启用CDN模式，都缓存本地静态资源
workbox.routing.registerRoute(
  /\.(?:js|css|png|jpg|jpeg|svg|gif|ico|woff|woff2|eot|ttf|otf)$/,
  new workbox.strategies.StaleWhileRevalidate({
    cacheName: 'static-resources',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxAgeSeconds: 7 * 24 * 60 * 60, // 7天缓存
        maxEntries: 50, // 最多缓存50个文件
      }),
    ],
  })
);

// 缓存HTML页面
```



```
workbox.routing.registerRoute(
  /\.html$/,
  new workbox.strategies.NetworkFirst({
    cacheName: 'html-cache',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxAgeSeconds: 1 * 24 * 60 * 60, // 1天缓存
        maxEntries: 10, // 最多缓存10个HTML文件
      }),
    ],
  })
);

// 离线页面 - 使用更可靠的处理方式
workbox.routing.setCatchHandler(async ({ event }) => {
  // 根据请求类型返回适当的默认页面
  switch (event.request.destination) {
    case 'document':
      // 如果是网页请求, 返回离线页面
      return caches.match(OFFLINE_URL);
    default:
      // 所有其他请求返回错误
      return Response.error();
  }
});

// ===== 文件: src/router/index.js =====
import Vue from 'vue'
import VueRouter from 'vue-router'
import { canAccessRoute, readStoredUserInfo } from './access.mjs'

Vue.use(VueRouter)

const routes = [
  {
    path: '/',
    name: 'welcome',
    component: function () {
      return import('../views/login.vue')
    }
  },
  {
    path: '/role-config',
    name: 'RoleConfig',
    component: function () {
      return import('../views/roleConfig.vue')
    }
  },
  {
    path: '/voice-print',
    name: 'VoicePrint',
    component: function () {
      return import('../views/VoicePrint.vue')
    }
  },
  {
    path: '/login',
    name: 'login',
    component: function () {
      return import('../views/login.vue')
    }
  }
]
```

```
},
{
  path: '/home',
  name: 'home',
  component: function () {
    return import('../views/home.vue')
  }
},
{
  path: '/register',
  name: 'Register',
  component: function () {
    return import('../views/register.vue')
  }
},
{
  path: '/retrieve-password',
  name: 'RetrievePassword',
  component: function () {
    return import('../views/retrievePassword.vue')
  }
},
// 设备管理页面路由
{
  path: '/device-management',
  name: 'DeviceManagement',
  component: function () {
    return import('../views/DeviceManagement.vue')
  }
},
// 添加用户管理路由
{
  path: '/user-management',
  name: 'UserManagement',
  component: function () {
    return import('../views/UserManagement.vue')
  }
},
{
  path: '/model-config',
  name: 'ModelConfig',
  component: function () {
    return import('../views/ModelConfig.vue')
  }
},
{
  path: '/params-management',
  name: 'ParamsManagement',
  component: function () {
    return import('../views/ParamsManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '参数管理'
  }
},
{
  path: '/knowledge-base-management',
  name: 'KnowledgeBaseManagement',
  component: function () {
```

```
    return import('../views/KnowledgeBaseManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '知识库管理'
  }
},
{
  path: '/server-side-management',
  name: 'ServerSideManager',
  component: function () {
    return import('../views/ServerSideManager.vue')
  },
  meta: {
    requiresAuth: true,
    title: '服务端管理'
  }
},
{
  path: '/ota-management',
  name: 'OtaManagement',
  component: function () {
    return import('../views/OtaManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: 'OTA管理'
  }
},
{
  path: '/voice-resource-management',
  name: 'VoiceResourceManagement',
  component: function () {
    return import('../views/VoiceResourceManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '音色资源开通'
  }
},
{
  path: '/voice-clone-management',
  name: 'VoiceCloneManagement',
  component: function () {
    return import('../views/VoiceCloneManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '音色克隆管理'
  }
},
{
  path: '/dict-management',
  name: 'DictManagement',
  component: function () {
    return import('../views/DictManagement.vue')
  }
},
{
  path: '/provider-management',
```

```
    name: 'ProviderManagement',
    component: function () {
      return import('../views/ProviderManagement.vue')
    }
  },
  // 添加默认角色管理路由
  {
    path: '/agent-template-management',
    name: 'AgentTemplateManagement',
    component: function () {
      return import('../views/AgentTemplateManagement.vue')
    }
  },
  // 添加模板快速配置路由
  {
    path: '/template-quick-config',
    name: 'TemplateQuickConfig',
    component: function () {
      return import('../views/TemplateQuickConfig.vue')
    }
  },
  // 功能配置页面路由
  {
    path: '/feature-management',
    name: 'FeatureManagement',
    component: function () {
      return import('../views/FeatureManagement.vue')
    },
    meta: {
      requiresAuth: true,
      title: '功能配置'
    }
  },
  // 替换词管理
  {
    path: '/replacement-word-management',
    name: 'ReplacementWordManagement',
    component: function () {
      return import('../views/ReplacementWordManagement.vue')
    },
    meta: {
      requiresAuth: true,
      title: '替换词管理'
    }
  },
  // 通讯录管理页面路由
  {
    path: '/address-book-management',
    name: 'AddressBookManagement',
    component: function () {
      return import('../views/AddressBookManagement.vue')
    },
    meta: {
      requiresAuth: true,
      title: '通讯录管理'
    }
  },
  // ===== NFC 生产管理路由 =====
  {
    path: '/pdc-nfc/product-types',
```

```
name: 'NfcProductTypes',
component: function () {
  return import('../views/nfc/NfcProductTypeManagement.vue')
},
meta: {
  requiresAuth: true,
  requiresSuperAdmin: true,
  title: 'NFC 商品类型'
}
},
{
  path: '/pdc-nfc/batches',
  name: 'NfcBatches',
  component: function () {
    return import('../views/nfc/NfcBatchManagement.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC 批次管理'
  }
},
{
  path: '/pdc-nfc/scheme',
  name: 'NfcScheme',
  component: function () {
    return import('../views/nfc/NfcSchemeManagement.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC Scheme 任务'
  }
},
{
  path: '/pdc-nfc/write',
  name: 'NfcWrite',
  component: function () {
    return import('../views/nfc/NfcWriteJobManagement.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC 写卡任务'
  }
},
{
  path: '/pdc-nfc/manual-write/:jobId',
  name: 'NfcManualWrite',
  component: function () {
    return import('../views/nfc/NfcManualWrite.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC 手动写卡'
  }
},
{
  path: '/pdc-nfc/assets',
```

```
name: 'NfcAssets',
component: function () {
  return import('../views/nfc/NfcAssetManagement.vue')
},
meta: {
  requiresAuth: true,
  requiresSuperAdmin: true,
  title: 'NFC 资产管理'
}
},
{
  path: '/pdc-nfc/activation',
  name: 'NfcActivation',
  component: function () {
    return import('../views/nfc/NfcActivationManagement.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC 扫码激活'
  }
},
{
  path: '/pdc-nfc/audit',
  name: 'NfcAudit',
  component: function () {
    return import('../views/nfc/NfcOperationLogManagement.vue')
  },
  meta: {
    requiresAuth: true,
    requiresSuperAdmin: true,
    title: 'NFC 审计日志'
  }
},
// 故事引擎内容运营页面路由
{
  path: '/story-engine-management',
  name: 'StoryEngineManagement',
  component: function () {
    return import('../views/StoryEngineManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '故事引擎'
  }
},
// 用户反馈管理页面路由
{
  path: '/feedback-management',
  name: 'FeedbackManagement',
  component: function () {
    return import('../views/FeedbackManagement.vue')
  },
  meta: {
    requiresAuth: true,
    title: '反馈管理'
  }
},
]
const router = new VueRouter({
```

```
base: process.env.VUE_APP_PUBLIC_PATH || '/',
routes
})

// 全局处理重复导航, 改为刷新页面
const originalPush = VueRouter.prototype.push
VueRouter.prototype.push = function push(location) {
  return originalPush.call(this, location).catch(err => {
    if (err.name === 'NavigationDuplicated') {
      // 如果是重复导航, 刷新页面
      window.location.reload()
    } else {
      // 其他错误正常抛出
      throw err
    }
  })
}

// 需要登录才能访问的路由
const protectedRoutes = ['home', 'RoleConfig', 'DeviceManagement', 'UserManagement', 'ModelConfig', 'KnowledgeBaseManagement', 'KnowledgeFileUpload', 'AddressBookManagement']

// 路由守卫
router.beforeEach((to, from, next) => {
  // 检查是否是受保护的路线
  if (protectedRoutes.includes(to.name)) {
    // 从localStorage获取token
    const token = localStorage.getItem('token')
    if (!token) {
      // 未登录, 跳转到登录页
      next({ name: 'login', query: { redirect: to.fullPath } })
      return
    }
  }

  // NFC 路由使用 meta 驱动访问控制
  if (to.meta && (to.meta.requiresAuth || to.meta.requiresSuperAdmin)) {
    const token = localStorage.getItem('token')
    // 优先使用 Vuex 中的 userInfo, 刷新状态时回退到 localStorage
    const vuexUserInfo = router.app && router.app.$store ? router.app.$store.state.userInfo : null
    const userInfo = vuexUserInfo || readStoredUserInfo()
    const result = canAccessRoute(to, token, userInfo)
    if (!result.allowed) {
      next({ path: result.redirect, query: result.redirect === '/login' ? { redirect: to.fullPath } : {} })
    }
    return
  }

  next()
})

export default router

// ===== 文件: src/store/index.js =====
import { goToPage } from '@utils';
import Vue from 'vue';
import Vuex from 'vuex';
import Api from '../apis/api';
import Constant from '../utils/constant';
```

```
Vue.use(Vuex)

export default new Vuex.Store({
  state: {
    token: '',
    userInfo: {}, // 添加用户信息存储
    pubConfig: { // 添加公共配置存储
      version: '',
      beianIcpNum: 'null',
      beianGaNum: 'null',
      allowUserRegister: false,
      sm2PublicKey: ''
    }
  },
  getters: {
    getToken(state) {
      if (!state.token) {
        state.token = localStorage.getItem('token')
      }
      return state.token
    },
    getUserInfo(state) {
      return state.userInfo
    },
    getPubConfig(state) {
      return state.pubConfig
    }
  },
  mutations: {
    setToken(state, token) {
      state.token = token
      localStorage.setItem('token', token)
    },
    setUserInfo(state, userInfo) {
      state.userInfo = userInfo
      localStorage.setItem('userInfo', JSON.stringify(userInfo))
    },
    setPubConfig(state, config) {
      state.pubConfig = config
      localStorage.setItem('pubConfig', JSON.stringify(config))
    },
    clearAuth(state) {
      state.token = ''
      state.userInfo = {}
      localStorage.removeItem('token')
      localStorage.removeItem('userInfo')
    }
  },
  actions: {
    // 添加 logout action
    logout({ commit }) {
      return new Promise((resolve) => {
        commit('clearAuth')
        goToPage(Constant.PAGE.LOGIN, true);
      })
    },
    // 添加获取公共配置的 action
    fetchPubConfig({ commit }) {
      return new Promise((resolve) => {
        Api.user.getPubConfig(({ data }) => {
```



```

        if (data.code === 0) {
            commit('setPubConfig', data.data);
        }
        resolve();
    });
});
}
},
modules: {
}
})
// ===== 文件: src/apis/api.js =====
// 引入各个模块的请求
import admin from './module/admin.js'
import agent from './module/agent.js'
import device from './module/device.js'
import dict from './module/dict.js'
import model from './module/model.js'
import ota from './module/ota.js'
import timbre from './module/timbre.js'
import user from './module/user.js'
import voiceClone from './module/voiceClone.js'
import voiceResource from './module/voiceResource.js'
import knowledgeBase from './module/knowledgeBase.js'
import correctWord from './module/correctWord.js'
import addressBook from './module/addressBook.js'
import pdcNfc from './module/pdcNfc.js'
import storyEngine from './module/storyEngine.js'
import feedback from './module/feedback.js'

/**
 * 接口地址
 * 开发时自动读取使用.env.development文件
 * 编译时自动读取使用.env.production文件
 */
const DEV_API_SERVICE = process.env.VUE_APP_API_BASE_URL

/**
 * 根据开发环境返回接口url
 * @returns {string}
 */
export function getServiceUrl() {
    return DEV_API_SERVICE
}

/** request服务封装 */
export default {
    getServiceUrl,
    user,
    admin,
    agent,
    device,
    model,
    timbre,
    ota,
    dict,
    voiceResource,
    voiceClone,

```

```
knowledgeBase,
correctWord,
addressBook,
pdcNfc,
storyEngine,
feedback
}
// ===== 文件: src/apis/httpRequest.js =====
import Fly from 'flyio/dist/npm/fly';
import store from '../store/index';
import Constant from '../utils/constant';
import { goToPage, isNotNull, showDanger, showWarning } from '../utils/index';
import i18n from '../i18n/index';

const fly = new Fly()
// 设置超时
fly.config.timeout = 30000

/**
 * Request服务封装
 */
export default {
  sendRequest,
  reAjaxFun,
  clearRequestTime
}

function sendRequest() {
  return {
    _sucCallback: null,
    _failCallback: null,
    _networkFailCallback: null,
    _method: 'GET',
    _data: {},
    _header: { 'content-type': 'application/json; charset=utf-8' },
    _url: '',
    _responseType: undefined, // 新增响应类型字段
    'send'() {
      // 设置语言请求头
      const currentLang = i18n.locale;
      // 转换语言代码格式, 将zh_CN转换为zh-CN
      let acceptLanguage = currentLang.replace('_', '-');
      // 为英语添加默认地区代码
      if (acceptLanguage === 'en') {
        acceptLanguage = 'en-US';
      }
      this._header['Accept-Language'] = acceptLanguage;

      if (isNotNull(store.getters.getToken)) {
        this._header.Authorization = 'Bearer ' + (JSON.parse(store.getters.getToken)).token
      }

      // 打印请求信息
      fly.request(this._url, this._data, {
        method: this._method,
        headers: this._header,
        responseType: this._responseType
      }).then((res) => {
        const error = httpHandlerError(res, this._failCallback, this._networkFailCallback);
        if (error) {

```

```
        return
    }

    if (this._sucCallback) {
        this._sucCallback(res)
    }
}).catch((res) => {
    // 打印失败响应
    console.log('catch', res)
    httpHandlerError(res, this._failCallback, this._networkFailCallback)
})
return this
},
'success'(callback) {
    this._sucCallback = callback
    return this
},
'fail'(callback) {
    this._failCallback = callback
    return this
},
'networkFail'(callback) {
    this._networkFailCallback = callback
    return this
},
'url'(url) {
    if (url) {
        url = url.replaceAll('$', '/')
    }
    this._url = url
    return this
},
'data'(data) {
    this._data = data
    return this
},
'method'(method) {
    this._method = method
    return this
},
'header'(header) {
    this._header = header
    return this
},
'showLoading'(showLoading) {
    this._showLoading = showLoading
    return this
},
'async'(flag) {
    this.async = flag
},
// 新增类型设置方法
'type'(responseType) {
    this._responseType = responseType;
    return this;
}
}
}
```

/**

```
* Info 请求完成后返回信息
* failCallback 回调函数
* networkFailCallback 回调函数
*/
// 在错误处理函数中添加日志
function httpHandlerError(info, failCallback, networkFailCallback) {

  /** 请求成功，退出该函数 可以根据项目需求来判断是否请求成功。这里判断的是status为200的时候是成功 */
  let networkError = false
  if (info.status === 200) {
    if (info.data.code === 'success' || info.data.code === 0 || info.data.code === undefined) {
      return networkError
    } else if (info.data.code === 401) {
      store.commit('clearAuth');
      goToPage(Constant.PAGE.LOGIN, true);
      return true
    } else {
      // 直接使用后端返回的国际化消息
      let errorMessage = info.data.msg;

      if (failCallback) {
        failCallback(info)
      } else {
        showDanger(errorMessage)
      }
      return true
    }
  }
  if (networkFailCallback) {
    networkFailCallback(info)
  } else {
    showDanger(`网络请求出现了错误 【${info.status}】`)
  }
  return true
}

let requestTime = 0
let reAjaxSec = 2

function reAjaxFun(fn) {
  let nowTimeSec = new Date().getTime() / 1000
  if (requestTime === 0) {
    requestTime = nowTimeSec
  }
  let ajaxIndex = parseInt((nowTimeSec - requestTime) / reAjaxSec)
  if (ajaxIndex > 10) {
    showWarning('似乎无法连接服务器')
  } else {
    showWarning('正在连接服务器(' + ajaxIndex + ')')
  }
  if (ajaxIndex < 10 && fn) {
    setTimeout(() => {
      fn()
    }, reAjaxSec * 1000)
  }
}

function clearRequestTime() {
  requestTime = 0
}
```

```
// ===== 文件: src/apis/module/addressBook.js =====
import { getServiceUrl } from '../api';
import RequestService from '../httpRequest';

export default {
  /**
   * 获取设备通讯录列表
   */
  getAddressBookList(macAddress, callback) {
    RequestService.sendRequest()
      .url(`${getServiceUrl()}/device/address-book/${macAddress}`)
      .method('GET')
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail(() => {
        RequestService.reAjaxFun(() => {
          this.getAddressBookList(macAddress, callback);
        });
      }).send();
  },

  /**
   * 更新设备通讯录别名
   */
  updateAlias(data, callback) {
    RequestService.sendRequest()
      .url(`${getServiceUrl()}/device/address-book/alias`)
      .method('PUT')
      .data(data)
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail(() => {
        RequestService.reAjaxFun(() => {
          this.updateAlias(data, callback);
        });
      }).send();
  },

  /**
   * 更新设备通讯录权限
   */
  updatePermission(data, callback) {
    RequestService.sendRequest()
      .url(`${getServiceUrl()}/device/address-book/permission`)
      .method('PUT')
      .data(data)
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail(() => {
        RequestService.reAjaxFun(() => {
          this.updatePermission(data, callback);
        });
      }).send();
  }
}
```

```
};  
// ===== 文件: src/apis/module/admin.js =====  
import { getServiceUrl } from '../api';  
import RequestService from '../httpRequest';  
  
export default {  
  // 用户列表  
  getUserList(params, callback) {  
    const queryParams = new URLSearchParams({  
      page: params.page,  
      limit: params.limit,  
      mobile: params.mobile  
    }).toString();  
  
    RequestService.sendRequest()  
      .url(`${getServiceUrl()}/admin/users?${queryParams}`)  
      .method('GET')  
      .success((res) => {  
        RequestService.clearRequestTime()  
        callback(res)  
      })  
      .networkFail((err) => {  
        console.error('请求失败:', err)  
        RequestService.reAjaxFun(() => {  
          this.getUserList(callback)  
        })  
      })  
      .send()  
  },  
  // 删除用户  
  deleteUser(id, callback) {  
    RequestService.sendRequest()  
      .url(`${getServiceUrl()}/admin/users/${id}`)  
      .method('DELETE')  
      .success((res) => {  
        RequestService.clearRequestTime()  
        callback(res)  
      })  
      .networkFail((err) => {  
        console.error('删除失败:', err)  
        RequestService.reAjaxFun(() => {  
          this.deleteUser(id, callback)  
        })  
      })  
      .send()  
  },  
  // 重置用户密码  
  resetUserPassword(id, callback) {  
    RequestService.sendRequest()  
      .url(`${getServiceUrl()}/admin/users/${id}`)  
      .method('PUT')  
      .success((res) => {  
        RequestService.clearRequestTime()  
        callback(res)  
      })  
      .networkFail((err) => {  
        console.error('重置密码失败:', err)  
        RequestService.reAjaxFun(() => {  
          this.resetUserPassword(id, callback)  
        })  
      })  
      .send()  
  }  
}
```

```
},
// 获取参数列表
getParamsList(params, callback) {
  const queryParams = new URLSearchParams({
    page: params.page,
    limit: params.limit,
    paramCode: params.paramCode || ''
  }).toString();

  RequestService.sendRequest()
    .url(`${getServiceUrl()}/admin/params/page?${queryParams}`)
    .method('GET')
    .success((res) => {
      RequestService.clearRequestTime()
      callback(res)
    })
    .networkFail((err) => {
      console.error('获取参数列表失败:', err)
      RequestService.reAjaxFun(() => {
        this.getParamsList(params, callback)
      })
    })
    .send()
},
// 保存
addParam(data, callback, failCallback) {
  RequestService.sendRequest()
    .url(`${getServiceUrl()}/admin/params`)
    .method('POST')
    .data(data)
    .success((res) => {
      RequestService.clearRequestTime()
      callback(res)
    })
    .fail((err) => {
      RequestService.clearRequestTime()
      failCallback(err)
    })
    .networkFail((err) => {
      console.error('添加参数失败:', err)
      RequestService.reAjaxFun(() => {
        this.addParam(data, callback)
      })
    })
    .send()
},
// 修改
updateParam(data, callback, failCallback) {
  RequestService.sendRequest()
    .url(`${getServiceUrl()}/admin/params`)
    .method('PUT')
    .data(data)
    .success((res) => {
      RequestService.clearRequestTime()
      callback(res)
    })
    .fail((err) => {
      RequestService.clearRequestTime()
      failCallback(err)
    })
    .networkFail((err) => {
      console.error('更新参数失败:', err)
    })
}
```

```

        RequestService.reAjaxFun(() => {
            this.updateParam(data, callback)
        })
    }).send()
},
// 删除
deleteParam(ids, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/admin/params/delete`)
        .method('POST')
        .data(ids)
        .success((res) => {
            RequestService.clearRequestTime()
            callback(res);
        })
        .networkFail((err) => {
            console.error('删除参数失败:', err)
            RequestService.reAjaxFun(() => {
                this.deleteParam(ids, callback)
            })
        })
    }).send()
},
// 获取ws服务端列表
getWsServerList(params, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/admin/server/server-list`)
        .method('GET')
        .success((res) => {
            RequestService.clearRequestTime()
            callback(res)
        })
        .networkFail((err) => {
            console.error('获取ws服务端列表失败:', err)
            RequestService.reAjaxFun(() => {
                this.getWsServerList(params, callback)
            })
        })
    }).send();
},
// 发送ws服务器动作指令
sendWsServerAction(data, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/admin/server/emit-action`)
        .method('POST')
        .data(data)
        .success((res) => {
            RequestService.clearRequestTime()
            callback(res)
        })
        .networkFail((err) => {
            RequestService.reAjaxFun(() => {
                this.sendWsServerAction(data, callback)
            })
        })
    }).send();
}
}

// ===== 文件: src/apis/module/agent.js =====
import { getServiceUrl } from '../api';
import RequestService from '../httpRequest';

```



```
const CALLBACK_RETRY_LIMIT = 10;
const CALLBACK_RETRY_DELAY_MS = 2000;
const CALLBACK_RETRY_WINDOW_MS = CALLBACK_RETRY_LIMIT * CALLBACK_RETRY_DELAY_MS;

function retryCallbackRequest(retry, retryCount, onTerminalFailure, error, retryStartedAt) {
  if (!onTerminalFailure) {
    RequestService.reAjaxFun(() => retry(retryCount + 1))
    return
  }
  const startedAt = retryStartedAt || Date.now()
  if (retryCount >= CALLBACK_RETRY_LIMIT || Date.now() - startedAt >= CALLBACK_RETRY_WINDOW_MS) {
    RequestService.clearRequestTime()
    onTerminalFailure(error)
    return
  }
  setTimeout(() => retry(retryCount + 1, startedAt), CALLBACK_RETRY_DELAY_MS)
}

function attachTerminalFailure(request, onTerminalFailure) {
  if (onTerminalFailure) {
    request.fail((error) => {
      RequestService.clearRequestTime()
      onTerminalFailure(error)
    })
  }
  return request
}

function terminateCallbackRequest(onTerminalFailure, error) {
  RequestService.clearRequestTime()
  if (onTerminalFailure) {
    onTerminalFailure(error)
  }
}

export default {
  // 获取智能体列表
  getAgentList(callback) {
    RequestService.sendRequest()
      .url(`${getServiceUrl()}/agent/list`)
      .method('GET')
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail(() => {
        RequestService.reAjaxFun(() => {
          this.getAgentList(callback);
        });
      }).send();
  },
  // 添加智能体
  addAgent(agentName, callback) {
    RequestService.sendRequest()
      .url(`${getServiceUrl()}/agent`)
      .method('POST')
      .data({ agentName: agentName })
      .success((res) => {
        RequestService.clearRequestTime();
      })
  }
}
```

```

        callback(res);
    })
    .networkFail(() => {
        RequestService.reAjaxFun(() => {
            this.addAgent(agentName, callback);
        });
    }).send();
},
// 删除智能体
deleteAgent(agentId, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}`)
        .method('DELETE')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail(() => {
            RequestService.reAjaxFun(() => {
                this.deleteAgent(agentId, callback);
            });
        }).send();
},
// 获取智能体配置
getDeviceConfig(agentId, callback, onTerminalFailure, retryCount = 0, retryStartedAt = 0) {
    const retryWindowStartedAt = retryStartedAt || Date.now()
    const request = RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}`)
        .method('GET')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail((err) => {
            console.error('获取配置失败:', err);
            retryCallbackRequest(
                (nextRetryCount, nextRetryStartedAt) => this.getDeviceConfig(
                    agentId,
                    callback,
                    onTerminalFailure,
                    nextRetryCount,
                    nextRetryStartedAt
                ),
                retryCount,
                onTerminalFailure,
                err,
                retryWindowStartedAt
            )
        })
    attachTerminalFailure(request, onTerminalFailure).send();
},
// 配置智能体
updateAgentConfig(agentId, configData, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}`)
        .method('PUT')
        .data(configData)
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })

```

```

    })
    .networkFail(() => {
      RequestService.reAjaxFun(() => {
        this.updateAgentConfig(agentId, configData, callback);
      });
    }).send();
  },
  // 获取智能体配置快照列表
  getAgentSnapshots(agentId, params, callback, onTerminalFailure, retryCount = 0, retryStartedAt = 0)
  {
    const retryWindowStartedAt = retryStartedAt || Date.now()
    const request = RequestService.sendRequest()
      .url(`${getServiceUrl()}/agent/${agentId}/snapshots`)
      .method('GET')
      .data(params)
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail((error) => {
        retryCallbackRequest(
          (nextRetryCount, nextRetryStartedAt) => this.getAgentSnapshots(
            agentId,
            params,
            callback,
            onTerminalFailure,
            nextRetryCount,
            nextRetryStartedAt
          ),
          retryCount,
          onTerminalFailure,
          error,
          retryWindowStartedAt
        )
      })
    attachTerminalFailure(request, onTerminalFailure).send();
  },
  // 获取智能体配置快照详情
  getAgentSnapshot(agentId, snapshotId, callback, onTerminalFailure, retryCount = 0, retryStartedAt = 0)
  {
    const retryWindowStartedAt = retryStartedAt || Date.now()
    const request = RequestService.sendRequest()
      .url(`${getServiceUrl()}/agent/${agentId}/snapshots/${snapshotId}`)
      .method('GET')
      .success((res) => {
        RequestService.clearRequestTime();
        callback(res);
      })
      .networkFail((error) => {
        retryCallbackRequest(
          (nextRetryCount, nextRetryStartedAt) => this.getAgentSnapshot(
            agentId,
            snapshotId,
            callback,
            onTerminalFailure,
            nextRetryCount,
            nextRetryStartedAt
          ),
          retryCount,
          onTerminalFailure,

```

```

        error,
        retryWindowStartedAt
    )
    })
    attachTerminalFailure(request, onTerminalFailure).send();
},
// 恢复智能体配置快照
restoreAgentSnapshot(agentId, snapshotId, currentStateToken, callback, onTerminalFailure) {
    const request = RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}/snapshots/${snapshotId}/restore`)
        .method('POST')
        .data({ currentStateToken })
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail((error) => {
            terminateCallbackRequest(onTerminalFailure, error)
        })
    attachTerminalFailure(request, onTerminalFailure).send();
},
// 删除智能体配置快照
deleteAgentSnapshot(agentId, snapshotId, callback, onTerminalFailure) {
    const request = RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}/snapshots/${snapshotId}`)
        .method('DELETE')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail((error) => {
            terminateCallbackRequest(onTerminalFailure, error)
        })
    attachTerminalFailure(request, onTerminalFailure).send();
},
// 新增方法：获取智能体模板
getAgentTemplate(callback) { // 移除templateName参数
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/template`)
        .method('GET')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail((err) => {
            console.error('获取模板失败:', err);
            RequestService.reAjaxFun(() => {
                this.getAgentTemplate(callback);
            });
        }).send();
},

// 新增：获取智能体模板分页列表
getAgentTemplatesPage(params, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/template/page`)
        .method('GET')
        .data(params)
        .success((res) => {
            RequestService.clearRequestTime();

```

```
        callback(res);
    })
    .networkFail((err) => {
        console.error('获取模板分页列表失败:', err);
        RequestService.reAjaxFun(() => {
            this.getAgentTemplatesPage(params, callback);
        });
    }).send();
},
// 获取智能体会话列表
getAgentSessions(agentId, params, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}/sessions`)
        .method('GET')
        .data(params)
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail(() => {
            RequestService.reAjaxFun(() => {
                this.getAgentSessions(agentId, params, callback);
            });
        }).send();
},
// 获取智能体聊天记录
getAgentChatHistory(agentId, sessionId, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/${agentId}/chat-history/${sessionId}`)
        .method('GET')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail(() => {
            RequestService.reAjaxFun(() => {
                this.getAgentChatHistory(agentId, sessionId, callback);
            });
        }).send();
},
// 获取音频下载ID
getAudioId(audioId, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/audio/${audioId}`)
        .method('POST')
        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .networkFail(() => {
            RequestService.reAjaxFun(() => {
                this.getAudioId(audioId, callback);
            });
        }).send();
},
// 获取智能体的MCP接入点地址
getAgentMcpAccessAddress(agentId, callback) {
    RequestService.sendRequest()
        .url(`${getServiceUrl()}/agent/mcp/address/${agentId}`)
        .method('GET')
```

```

        .success((res) => {
            RequestService.clearRequestTime();
            callback(res);
        })
        .fail((err) => {
            callback(err);
        })
        .networkFail(() => {
            RequestService.reAjaxFun(() => {
                this.getAgentMcpAccessAddress(agentId, callback);
            });
        }).send();
    },
    // 获取智能体的MCP工具列表
    getAgentMcpToolsList(agentId, callback) {
        RequestService.sendRequest()
            .url(`${getServiceUrl()}/agent/mcp/tools/${agentId}`)
            .method('GET')
            .success((res) => {
                RequestService.clearRequestTime();
                callback(res);
            })
            .networkFail(() => {
                RequestService.reAjaxFun(() => {
                    this.getAgentMcpToolsList(agentId, callback);
                });
            }).send();
    },
    // 添加智能体的声纹
    addAgentVoicePrint(voicePrintData, callback) {
        RequestService.sendRequest()
            .url(`${getServiceUrl()}/agent/voice-print`)
            .method('POST')
            .data(voicePrintData)
            .success((res) => {
                RequestService.clearRequestTime();
                callback(res);
            })
            .networkFail(() => {
                RequestService.reAjaxFun(() => {
                    this.addAgentVoicePrint(voicePrintData, callback);
                });
            }).send();
    },
    // 获取指定智能体声纹列表
    getAgentVoicePrintList(id, callback) {
        RequestService.sendRequest()
            .url(`${getServiceUrl()}/agent/voice-print/list/${id}`)
            .method('GET')
            .success((res) => {
                RequestService.clearRequestTime();
                callback(res);
            })
            .networkFail(() => {
                RequestService.reAjaxFun(() => {
                    this.getAgentVoicePrintList(id, callback);
                });
            }).send();
    },
    // 删除智能体声纹

```

```
      this.logsPage = page
      this.fetchLogs()
    },
    handleClose() {
      this.dialogVisible = false
      this.rawAsset = null
      this.logsData = []
      this.logsTotal = 0
      this.logsPage = 1
    }
  }
}
</script>

<style lang="scss" scoped>
.asset-detail {
  min-height: 120px;
}

.hash-text {
  font-family: monospace;
  font-size: 13px;
  word-break: break-all;
}

.section-title {
  font-size: 15px;
  font-weight: 500;
  margin: 20px 0 12px;
  color: #303133;
}

.timeline-section {
  margin-top: 8px;
}

.logs-section {
  margin-top: 8px;
}

.logs-pagination {
  display: flex;
  justify-content: flex-end;
  padding-top: 10px;
}
</style>
// ===== 文件: src/components/nfc/NfcBatchDialog.vue =====
<template>
  <el-dialog
    title="创建批次"
    :visible.sync="dialogVisible"
    width="600px"
    :close-on-click-modal="false"
    @close="handleClose"
  >
    <el-form
      ref="batchForm"
      :model="form"
      :rules="rules"
      label-width="100px"
    >
```

```
label-position="right"
>
<el-form-item label="批次号" prop="batchNo">
  <el-input
    v-model="form.batchNo"
    placeholder="全局唯一，资产编号将以此为前缀"
    maxlength="64"
  ></el-input>
</el-form-item>

<el-form-item label="商品类型" prop="productId">
  <el-select
    v-model="form.productId"
    placeholder="请选择商品类型"
    filterable
    style="width: 100%;"
    :loading="productTypeLoading"
  >
    <el-option
      v-for="item in productTypeOptions"
      :key="item.id"
      :label="`${item.typeCode} - ${item.typeName}`"
      :value="item.id"
    ></el-option>
  </el-select>
</el-form-item>

<el-form-item label="SKU 编码" prop="skuCode">
  <el-input
    v-model="form.skuCode"
    placeholder="请输入 SKU 编码"
    maxlength="64"
  ></el-input>
</el-form-item>

<el-form-item label="原型" prop="prototype">
  <el-select
    v-model="form.prototype"
    placeholder="请选择原型"
    style="width: 100%;"
    :loading="prototypeLoading"
  >
    <el-option
      v-for="item in prototypeOptions"
      :key="item.key"
      :label="item.name"
      :value="item.key"
    ></el-option>
  </el-select>
</el-form-item>

<el-form-item label="计划数量" prop="plannedQuantity">
  <el-input-number
    v-model="form.plannedQuantity"
    :min="1"
    :max="10000"
    :step="1"
    controls-position="right"
  ></el-input-number>
</el-form-item>
```



```

    <el-form-item label="备注" prop="remark">
      <el-input
        type="textarea"
        v-model="form.remark"
        :rows="3"
        placeholder="请输入备注（可选）"
        maxlength="200"
        show-word-limit
      ></el-input>
    </el-form-item>
  </el-form>

  <template slot="footer">
    <el-button @click="handleClose">取消</el-button>
    <el-button type="primary" :loading="saving" @click="handleSubmit">确定</el-button>
  </template>
</el-dialog>
</template>

<script>
import Api from '@apis/api'

export default {
  name: 'NfcBatchDialog',
  props: {
    visible: {
      type: Boolean,
      default: false
    }
  },
  data() {
    return {
      dialogVisible: this.visible,
      saving: false,
      productTypeLoading: false,
      productTypeOptions: [],
      prototypeLoading: false,
      prototypeOptions: [],
      form: {
        batchNo: '',
        productTypeId: '',
        skuCode: '',
        prototype: '',
        plannedQuantity: 1,
        remark: ''
      },
      rules: {
        batchNo: [
          { required: true, message: '请输入批次号', trigger: 'blur' },
          { max: 64, message: '批次号长度不能超过 64 个字符', trigger: 'blur' }
        ],
        productTypeId: [
          { required: true, message: '请选择商品类型', trigger: 'change' }
        ],
        skuCode: [
          { required: true, message: '请输入 SKU 编码', trigger: 'blur' },
          { max: 64, message: 'SKU 编码长度不能超过 64 个字符', trigger: 'blur' }
        ],
        prototype: [

```

```

      { required: true, message: '请选择原型', trigger: 'change' }
    ],
    plannedQuantity: [
      { required: true, message: '请输入计划数量', trigger: 'blur' },
      { type: 'number', min: 1, max: 10000, message: '数量范围 1-10000', trigger: 'blur' }
    ]
  }
},
watch: {
  visible(val) {
    this.dialogVisible = val
    if (val) {
      this.fetchProductTypes()
      this.fetchPrototypes()
    }
  },
  dialogVisible(val) {
    this.$emit('update:visible', val)
  }
},
methods: {
  fetchProductTypes() {
    this.productTypeLoading = true
    Api.pdcNfc.listProductTypes({ page: 1, limit: 200 }, (res) => {
      this.productTypeLoading = false
      if (res.data && res.data.code === 0) {
        this.productTypeOptions = res.data.data?.list || res.data.data || []
      } else {
        this.$message.error(res.data?.msg || '获取商品类型列表失败')
      }
    })
  },
  // 原型选项来自字典 EGG_PET_PROTOTYPE, dict_value 为中文且与后端校验值一致
  // 拉取失败时不做本地兜底：宁可下拉为空暴露异常，也不能用可能过期的硬编码值生产错误资产
  fetchPrototypes() {
    this.prototypeLoading = true
    Api.dict.getDictDataByType('EGG_PET_PROTOTYPE')
      .then((data) => {
        this.prototypeLoading = false
        this.prototypeOptions = Array.isArray(data) ? data : (data.data || [])
      })
      .catch(() => {
        this.prototypeLoading = false
        this.prototypeOptions = []
        this.$message.error('获取原型字典失败，请稍后重试')
      })
  },
  handleSubmit() {
    this.$refs.batchForm.validate((valid) => {
      if (!valid) return
      this.saving = true
      Api.pdcNfc.createBatch(this.form, (res) => {
        this.saving = false
        if (res.data && res.data.code === 0) {
          this.$message.success('批次创建成功')
          this.$emit('created')
          this.handleClose()
        } else {
          this.$message.error(res.data?.msg || '批次创建失败')
        }
      })
    })
  }
}

```

```

    }
  })
})
},
handleClose() {
  this.dialogVisible = false
  this.saving = false
  this.$nextTick(() => {
    if (this.$refs.batchForm) {
      this.$refs.batchForm.resetFields()
    }
    this.form = {
      batchNo: '',
      productTypeId: '',
      skuCode: '',
      prototype: '',
      plannedQuantity: 1,
      remark: ''
    }
  })
}
}
}
}
</script>
// ===== 文件: src/components/nfc/NfcStockInDialog.vue =====
<template>
  <el-dialog
    title="资产批量入库"
    :visible.sync="dialogVisible"
    width="600px"
    :close-on-click-modal="false"
    @close="handleClose"
  >
    <!-- 步骤 1: 填写表单 -->
    <div v-if="step === 'form'" class="stock-in-form">
      <el-form
        ref="stockInForm"
        :model="form"
        :rules="rules"
        label-width="110px"
        label-position="right"
      >
        <el-form-item label="入库资产">
          <div class="asset-summary">
            <el-tag type="info">共 {{ assetIds.length }} 项</el-tag>
            <span v-if="assetIds.length === 0" class="text-muted">请在资产列表中勾选需要入库的资产</span>
          </div>
        </el-form-item>

        <el-form-item label="入库单号" prop="businessNo">
          <el-input
            v-model="form.businessNo"
            placeholder="请输入入库业务单号"
            maxlength="64"
          ></el-input>
        </el-form-item>

        <el-form-item label="备注" prop="remark">
          <el-input

```

```
        type="textarea"
        v-model="form.remark"
        :rows="3"
        placeholder="请输入备注（可选）"
        maxlength="200"
        show-word-limit
      ></el-input>
    </el-form-item>
  </el-form>
</div>

<!-- 步骤 2: 确认 -->
<div v-if="step === 'confirm'" class="confirm-section">
  <el-alert title="请确认入库信息" type="info" :closable="false" show-icon></el-alert>
  <el-descriptions :column="1" border size="medium" class="confirm-desc">
    <el-descriptions-item label="入库资产数">{{ assetIds.length }} 项</el-descriptions-item>
    <el-descriptions-item label="入库单号">{{ form.businessNo }}</el-descriptions-item>
    <el-descriptions-item label="备注">{{ form.remark || '-' }}</el-descriptions-item>
    <el-descriptions-item label="请求 ID">{{ form.requestId }}</el-descriptions-item>
  </el-descriptions>
</div>

<!-- 步骤 3: 结果 -->
<div v-if="step === 'result'" class="result-section">
  <el-result
    :icon="resultIcon"
    :title="resultTitle"
    :subTitle="resultSubTitle"
  >
    <template slot="extra">
      <el-button type="primary" @click="handleClose">完成</el-button>
    </template>
  </el-result>
  <div v-if="result" class="result-detail">
    <el-descriptions :column="2" border size="medium">
      <el-descriptions-item label="处理数量">{{ result.processedCount ?? 0 }}</el-descriptions-item>
      <el-descriptions-item label="成功数量">{{ result.successCount ?? 0 }}</el-descriptions-item>
    </el-descriptions>
  </div>
</div>

<template slot="footer">
  <template v-if="step === 'form'">
    <el-button @click="handleClose">取消</el-button>
    <el-button
      type="primary"
      :disabled="assetIds.length === 0"
      @click="goConfirm"
    >下一步</el-button>
  </template>
  <template v-if="step === 'confirm'">
    <el-button @click="step = 'form'">上一步</el-button>
    <el-button type="primary" :loading="submitting" @click="handleSubmit">确认入库</el-button>
  </template>
</template>
</el-dialog>
</template>

<script>
import Api from '@apis/api'
```

```

export default {
  name: 'NfcStockInDialog',
  props: {
    visible: {
      type: Boolean,
      default: false
    },
    assetIds: {
      type: Array,
      default: () => []
    }
  },
  data() {
    return {
      dialogVisible: this.visible,
      step: 'form',
      submitting: false,
      result: null,
      form: {
        businessNo: '',
        remark: '',
        requestId: ''
      },
      rules: {
        businessNo: [
          { required: true, message: '请输入入库业务单号', trigger: 'blur' },
          { max: 64, message: '单号长度不能超过 64 个字符', trigger: 'blur' }
        ]
      }
    }
  },
  computed: {
    resultIcon() {
      if (this.result && this.result.successCount === 0) return 'error'
      if (this.result && this.result.successCount < (this.result.processedCount || 0)) return 'warning'
      return 'success'
    },
    resultTitle() {
      if (this.result && this.result.successCount === 0) return '入库失败'
      if (this.result && this.result.successCount < (this.result.processedCount || 0)) return '部分成功'
      return '入库成功'
    },
    resultSubTitle() {
      if (!this.result) return ''
      return `处理 ${this.result.processedCount ?? 0} 项, 成功 ${this.result.successCount ?? 0} 项`
    }
  },
  watch: {
    visible(val) {
      this.dialogVisible = val
    },
    dialogVisible(val) {
      this.$emit('update:visible', val)
    }
  },
  methods: {
    generateUUID() {
      if (typeof crypto !== 'undefined' && crypto.randomUUID) {
        return crypto.randomUUID()
      }
    }
  }
}

```

```

    }
    return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, (c) => {
      const r = Math.random() * 16 | 0
      const v = c === 'x' ? r : (r & 0x3 | 0x8)
      return v.toString(16)
    })
  },
  goConfirm() {
    this.$refs.stockInForm.validate((valid) => {
      if (!valid) return
      this.form.requestId = this.generateUUID()
      this.step = 'confirm'
    })
  },
  handleSubmit() {
    this.submitting = true
    const payload = {
      assetIds: [...this.assetIds],
      businessNo: this.form.businessNo,
      requestId: this.form.requestId
    }
    if (this.form.remark) payload.remark = this.form.remark
    Api.pdcNfc.stockIn(payload, (res) => {
      this.submitting = false
      if (res.data && res.data.code === 0) {
        this.result = res.data.data || {}
        this.step = 'result'
        this.$emit('stocked', this.result)
      } else {
        this.$message.error(res.data?.msg || '入库失败')
      }
    })
  },
  handleClose() {
    this.dialogVisible = false
    this.step = 'form'
    this.result = null
    this.form = { businessNo: '', remark: '', requestId: '' }
    this.$nextTick(() => {
      if (this.$refs.stockInForm) {
        this.$refs.stockInForm.resetFields()
      }
    })
  }
}
}
</script>

<style lang="scss" scoped>
.asset-summary {
  display: flex;
  align-items: center;
  gap: 8px;
}

.text-muted {
  color: #c0c4cc;
  font-size: 13px;
}

```

```

.confirm-section {
  .confirm-desc {
    margin-top: 16px;
  }
}

.result-detail {
  margin-top: 16px;
}
</style>
// ===== 文件: src/components/nfc/NfcWriteJobDialog.vue =====
<template>
  <el-dialog
    title="写卡任务"
    :visible.sync="dialogVisible"
    width="650px"
    :close-on-click-modal="false"
    @close="handleClose"
  >
    <div v-loading="loading" class="write-job-detail">
      <el-descriptions :column="2" border size="medium">
        <el-descriptions-item label="任务ID">{{ jobDetail.id || '-' }}</el-descriptions-item>
        <el-descriptions-item label="批次号">{{ jobDetail.batchNo || '-' }}</el-descriptions-item>
        <el-descriptions-item label="状态">
          <el-tag :type="badgeType(jobDetail.status)">{{ statusText(jobDetail.status) }}</el-tag>
        </el-descriptions-item>
        <el-descriptions-item label="计划数量">{{ jobDetail.plannedQuantity || '-' }}</el-descriptions-i
tem>
        <el-descriptions-item label="已导出">{{ jobDetail.exportedCount ?? 0 }}</el-descriptions-item>
        <el-descriptions-item label="已写入">{{ jobDetail.writtenCount ?? 0 }}</el-descriptions-item>
        <el-descriptions-item label="已验证">{{ jobDetail.verifiedCount ?? 0 }}</el-descriptions-item>
        <el-descriptions-item label="失败数">{{ jobDetail.failureCount ?? 0 }}</el-descriptions-item>
        <el-descriptions-item label="创建时间" :span="2">{{ formatTime(jobDetail.createTime) }}</el-desc
riptions-item>
      </el-descriptions>

      <div v-if="jobDetail.status === 'EXPORTED' || jobDetail.status === 'RESULT_IMPORTED'" class="job-a
ctions">
        <el-button type="primary" icon="el-icon-download" :loading="downloading" @click="handleDownload"
        >
          下载写卡数据
        </el-button>
        <el-button type="warning" icon="el-icon-upload2" @click="showImportDialog = true">
          导入写卡结果
        </el-button>
      </div>
    </div>

    <template slot="footer">
      <el-button @click="handleClose">关闭</el-button>
    </template>

    <NfcWriteResultImportDialog
      :visible.sync="showImportDialog"
      :job-id="jobDetail.id || jobId"
      @imported="onImported"
    />
  </el-dialog>
</template>

```

```
<script>
import Api from '@apis/api'
import { statusBadgeType, statusLabel, formatDate } from '@utils/pdcNfcState.mjs'
import NfcWriteResultImportDialog from './NfcWriteResultImportDialog.vue'

export default {
  name: 'NfcWriteJobDialog',
  components: { NfcWriteResultImportDialog },
  props: {
    visible: {
      type: Boolean,
      default: false
    },
    jobId: {
      type: [String, Number],
      default: ''
    }
  },
  data() {
    return {
      dialogVisible: this.visible,
      loading: false,
      downloading: false,
      showImportDialog: false,
      jobDetail: {},
      pollTimer: null
    }
  },
  watch: {
    visible(val) {
      this.dialogVisible = val
      if (val && this.jobId) {
        this.fetchJobDetail()
        this.startPolling()
      }
    },
    dialogVisible(val) {
      this.$emit('update:visible', val)
      if (!val) {
        this.stopPolling()
      }
    },
    jobId(val) {
      if (this.dialogVisible && val) {
        this.fetchJobDetail()
      }
    }
  },
  beforeDestroy() {
    this.stopPolling()
  },
  methods: {
    badgeType: statusBadgeType,
    statusText: statusLabel,
    formatTime: formatDate,
    fetchJobDetail() {
      if (!this.jobId) return
      this.loading = true
      Api.pdcNfc.getWriteJob(this.jobId, (res) => {
        this.loading = false
      })
    }
  }
}
```



```

    if (res.data && res.data.code === 0) {
      this.jobDetail = res.data.data || {}
      // Stop polling if terminal state
      if (['EXPORTED', 'RESULT_IMPORTED', 'FAILED', 'CANCELLED'].includes(this.jobDetail.status)) {
        this.stopPolling()
      }
    } else {
      this.$message.error(res.data?.msg || '获取写卡任务详情失败')
    }
  })
},
startPolling() {
  this.stopPolling()
  this.pollTimer = setInterval(() => {
    this.fetchJobDetail()
  }, 3000)
},
stopPolling() {
  if (this.pollTimer) {
    clearInterval(this.pollTimer)
    this.pollTimer = null
  }
},
handleDownload() {
  if (!this.jobDetail.id) {
    this.$message.warning('任务信息不完整')
    return
  }
  this.downloading = true
  Api.pdcNfc.downloadWriteJob(this.jobDetail.id, (res) => {
    this.downloading = false
    // The blob response comes back as res.data (already a Blob when responseType is blob)
    const blob = res.data instanceof Blob ? res.data : new Blob([res.data])
    const url = window.URL.createObjectURL(blob)
    const link = document.createElement('a')
    link.href = url
    link.download = `write_job_${this.jobDetail.id}.csv`
    document.body.appendChild(link)
    link.click()
    document.body.removeChild(link)
    window.URL.revokeObjectURL(url)
    this.$message.success('下载已开始')
  })
},
onImported() {
  this.fetchJobDetail()
  this.$emit('refresh')
},
handleClose() {
  this.dialogVisible = false
  this.jobDetail = {}
}
}
}
</script>

<style lang="scss" scoped>
.write-job-detail {
  min-height: 120px;
}

```

```

.job-actions {
  margin-top: 20px;
  display: flex;
  gap: 12px;
}
</style>
// ===== 文件: src/components/nfc/NfcWriteResultImportDialog.vue =====
<template>
  <el-dialog
    title="导入写卡结果"
    :visible.sync="dialogVisible"
    width="550px"
    :close-on-click-modal="false"
    @close="handleClose"
  >
    <el-form label-width="100px" label-position="right">
      <el-form-item label="写卡任务">
        <span>{{ jobId || '-' }}</span>
      </el-form-item>

      <el-form-item label="结果文件">
        <el-upload
          ref="upload"
          action="#"
          :auto-upload="false"
          :limit="1"
          :accept="'.csv'"
          :on-change="handleFileChange"
          :on-remove="handleFileRemove"
          :file-list="fileList"
        >
          <el-button size="small" type="primary">选择 CSV 文件</el-button>
          <div slot="tip" class="el-upload__tip">仅支持 .csv 格式的写卡结果文件</div>
        </el-upload>
      </el-form-item>

      <el-form-item v-if="importResult" label="导入结果">
        <div class="import-result">
          <el-tag type="success">已验证: {{ importResult.verifiedCount || 0 }}</el-tag>
          <el-tag type="warning" class="ml-8">已写入: {{ importResult.writtenCount || 0 }}</el-tag>
          <el-tag type="danger" class="ml-8">失败: {{ importResult.failureCount || 0 }}</el-tag>
        </div>
      </el-form-item>
    </el-form>

    <template slot="footer">
      <el-button @click="handleClose">关闭</el-button>
      <el-button
        type="primary"
        :loading="importing"
        :disabled="!selectedFile"
        @click="handleImport"
      >
        开始导入
      </el-button>
    </template>
  </el-dialog>
</template>

```

```

<script>
import Api from '@apis/api'

export default {
  name: 'NfcWriteResultImportDialog',
  props: {
    visible: {
      type: Boolean,
      default: false
    },
    jobId: {
      type: [String, Number],
      default: ''
    }
  },
  data() {
    return {
      dialogVisible: this.visible,
      importing: false,
      selectedFile: null,
      fileList: [],
      importResult: null
    }
  },
  watch: {
    visible(val) {
      this.dialogVisible = val
    },
    dialogVisible(val) {
      this.$emit('update:visible', val)
    }
  },
  methods: {
    generateUUID() {
      if (typeof crypto !== 'undefined' && crypto.randomUUID) {
        return crypto.randomUUID()
      }
      // Fallback UUID v4
      return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, (c) => {
        const r = Math.random() * 16 | 0
        const v = c === 'x' ? r : (r & 0x3 | 0x8)
        return v.toString(16)
      })
    },
    handleFileChange(file) {
      const name = file.name || ''
      if (!name.toLowerCase().endsWith('.csv')) {
        this.$message.error('请选择 .csv 格式的文件')
        this.$refs.upload.clearFiles()
        this.selectedFile = null
        return
      }
      this.selectedFile = file.raw
      this.importResult = null
    },
    handleFileRemove() {
      this.selectedFile = null
      this.importResult = null
    },
    handleImport() {

```

```

    if (!this.selectedFile) {
      this.$message.warning('请先选择结果文件')
      return
    }
    if (!this.jobId) {
      this.$message.error('缺少写卡任务 ID')
      return
    }
    this.importing = true
    const requestId = this.generateUUID()
    Api.pdcNfc.importWriteResult(this.jobId, this.selectedFile, requestId, (res) => {
      this.importing = false
      if (res.data && res.data.code === 0) {
        this.importResult = res.data.data || {}
        this.$message.success('导入成功')
        this.$emit('imported', this.importResult)
      } else {
        this.$message.error(res.data?.msg || '导入失败')
      }
    })
  },
  handleClose() {
    this.dialogVisible = false
    this.selectedFile = null
    this.fileList = []
    this.importResult = null
    this.$nextTick(() => {
      if (this.$refs.upload) {
        this.$refs.upload.clearFiles()
      }
    })
  }
}
}
</script>

<style lang="scss" scoped>
.import-result {
  display: flex;
  align-items: center;
}

.ml-8 {
  margin-left: 8px;
}
</style>

// ===== 文件: src/utils/cacheViewer.js =====
/**
 * 缓存查看工具 - 用于检查CDN资源是否已被Service Worker缓存
 */

/**
 * 获取所有Service Worker缓存的名称
 * @returns {Promise<string[]>} 缓存名称列表
 */
export const getCacheNames = async () => {
  if (!('caches' in window)) {
    return [];
  }

```

```

    try {
      return await caches.keys();
    } catch (error) {
      console.error('获取缓存名称失败:', error);
      return [];
    }
  };

/**
 * 获取指定缓存中的所有URL
 * @param {string} cacheName 缓存名称
 * @returns {Promise<string[]>} 缓存的URL列表
 */
export const getCacheUrls = async (cacheName) => {
  if (!('caches' in window)) {
    return [];
  }

  try {
    const cache = await caches.open(cacheName);
    const requests = await cache.keys();
    return requests.map(request => request.url);
  } catch (error) {
    console.error(`获取缓存 ${cacheName} 的URL失败:`, error);
    return [];
  }
};

/**
 * 检查特定URL是否已被缓存
 * @param {string} url 要检查的URL
 * @returns {Promise<boolean>} 是否已缓存
 */
export const isUrlCached = async (url) => {
  if (!('caches' in window)) {
    return false;
  }

  try {
    const cacheNames = await getCacheNames();
    for (const cacheName of cacheNames) {
      const cache = await caches.open(cacheName);
      const match = await cache.match(url);
      if (match) {
        return true;
      }
    }
    return false;
  } catch (error) {
    console.error(`检查URL ${url} 是否缓存失败:`, error);
    return false;
  }
};

/**
 * 获取当前页面所有CDN资源的缓存状态
 * @returns {Promise<Object>} 缓存状态对象
 */
export const checkCdnCacheStatus = async () => {
  // 从CDN缓存中查找资源

```

```
const cdnCaches = ['cdn-stylesheets', 'cdn-scripts'];
const results = {
  css: [],
  js: [],
  totalCached: 0,
  totalNotCached: 0
};

for (const cacheName of cdnCaches) {
  try {
    const urls = await getCacheUrls(cacheName);

    // 区分CSS和JS资源
    for (const url of urls) {
      if (url.endsWith('.css')) {
        results.css.push({ url, cached: true });
      } else if (url.endsWith('.js')) {
        results.js.push({ url, cached: true });
      }
      results.totalCached++;
    }
  } catch (error) {
    console.error(`获取 ${cacheName} 缓存信息失败:`, error);
  }
}

return results;
};

/**
 * 清除所有Service Worker缓存
 * @returns {Promise<boolean>} 是否成功清除
 */
export const clearAllCaches = async () => {
  if (!('caches' in window)) {
    return false;
  }

  try {
    const cacheNames = await getCacheNames();
    for (const cacheName of cacheNames) {
      await caches.delete(cacheName);
    }
    return true;
  } catch (error) {
    console.error('清除所有缓存失败:', error);
    return false;
  }
};

/**
 * 将缓存状态输出到控制台
 */
export const logCacheStatus = async () => {
  console.group('Service Worker 缓存状态');

  const cacheNames = await getCacheNames();
  console.log('已发现的缓存:', cacheNames);

  for (const cacheName of cacheNames) {
```

```

    const urls = await getCacheUrls(cacheName);
    console.group(`缓存: ${cacheName} (${urls.length} 项)`);
    urls.forEach(url => console.log(url));
    console.groupEnd();
  }

  console.groupEnd();
  return cacheNames.length > 0;
};

// ===== 文件: src/utils/constant.js =====
const HAVE_NO_RESULT = '暂无'
export default {
  HAVE_NO_RESULT, // 项目的配置信息
  PAGE: {
    LOGIN: '/login',
  },
  STORAGE_KEY: {
    TOKEN: 'TOKEN',
    PUBLIC_KEY: 'PUBLIC_KEY',
    USER_TYPE: 'USER_TYPE'
  },
  Lang: {
    'zh_cn': 'zh_cn', 'zh_tw': 'zh_tw', 'en': 'en'
  },
  FONT_SIZE: {
    'big': 'big',
    'normal': 'normal',
  }, // 获取map中的某key
  get(map, key) {
    return map[key] || HAVE_NO_RESULT
  }
}

// ===== 文件: src/utils/date.js =====
export const toDate = (date) => {
  return isDate(date) ? new Date(date) : null
}

export const isDate = (date) => {
  if (date === null || date === undefined) return false
  if (isNaN(new Date(date).getTime())) return false
  return true
}

export const isDateObject = (val) => {
  return val instanceof Date
}

export const formatAddDate = (date, format, addDay) => {
  date = toDate(date)
  if (!date) {
    return ''
  }
  if (!addDay) {
    date.setDate(date.getDate() + addDay)
  }
  return formatDateTool(date, format || 'yyyy-MM-dd HH:mm:ss')
}

export const formatDate = (date, format) => {
  date = toDate(date)

```

```

    if (!date) return ''
    return formatDateTool(date, format || 'yyyy-MM-dd HH:mm:ss')
}

function formatDateTool(date, fmt) {
    if (/(\w+)/.test(fmt)) {
        fmt = fmt.replace(
            RegExp.$1,
            (date.getFullYear() + '').substr(4 - RegExp.$1.length)
        )
    }
    const o = {
        'M+': date.getMonth() + 1,
        'd+': date.getDate(),
        'H+': date.getHours(),
        'h+': date.getHours(),
        'm+': date.getMinutes(),
        's+': date.getSeconds()
    }
    for (const k in o) {
        if (new RegExp(`(${k})`).test(fmt)) {
            const str = o[k] + ''
            fmt = fmt.replace(
                RegExp.$1,
                RegExp.$1.length === 1 ? str : padLeftZero(str)
            )
        }
    }
    return fmt
}

function padLeftZero(str) {
    return ('00' + str).substr(str.length)
}

// ===== 文件: src/utils/featureManager.js =====
//功能配置工具
import Api from "@apis/api";
import store from "@store";

class FeatureManager {
    constructor() {
        this.defaultFeatures = {
            voiceprintRecognition: {
                name: 'feature.voiceprintRecognition.name',
                enabled: false,
                description: 'feature.voiceprintRecognition.description'
            },
            voiceClone: {
                name: 'feature.voiceClone.name',
                enabled: false,
                description: 'feature.voiceClone.description'
            },
            knowledgeBase: {
                name: 'feature.knowledgeBase.name',
                enabled: false,
                description: 'feature.knowledgeBase.description'
            },
            mcpAccessPoint: {
                name: 'feature.mcpAccessPoint.name',
                enabled: false,

```



```

        description: 'feature.mcpAccessPoint.description'
      },
      vad: {
        name: 'feature.vad.name',
        enabled: false,
        description: 'feature.vad.description'
      },
      asr: {
        name: 'feature.asr.name',
        enabled: false,
        description: 'feature.asr.description'
      },
      addressBook: {
        name: 'feature.addressBook.name',
        enabled: false,
        description: 'feature.addressBook.description'
      }
    };
    this.currentFeatures = { ...this.defaultFeatures }; // 当前内存中的配置
    this.initialized = false;
    this.initPromise = null;
  }

  /**
   * 等待初始化完成
   */
  async waitForInitialization() {
    if (!this.initPromise) {
      this.initPromise = this.init();
    }
    await this.initPromise;
    return this.initialized;
  }

  /**
   * 初始化功能配置
   */
  async init() {
    try {
      // 从pub-config接口获取配置
      const config = await this.getConfigFromPubConfig();
      if (config) {
        this.currentFeatures = { ...config }; // 保存到内存
        this.initialized = true;
        return;
      }
    } catch (error) {
      console.warn('从pub-config接口获取配置失败:', error);
    }

    // pub-config接口失败，使用默认配置
    this.currentFeatures = { ...this.defaultFeatures }; // 保存默认配置到内存
    this.initialized = true;
  }

  /**
   * 更新config缓存
   */
  updateConfigCache(config) {
    store.commit('setPubConfig', config);
  }

```

```

    localStorage.setItem('pubConfig', JSON.stringify(config));
}

/**
 * 从pub-config接口获取配置
 */
async getConfigFromPubConfig() {
    return new Promise((resolve) => {
        // 直接调用pub-config接口获取配置
        Api.user.getPubConfig((result) => {
            // 检查返回结果的结构
            if (result && result.status === 200) {
                // 检查是否有data字段
                if (result.data) {
                    const configCache = result.data.data || {};
                    // 检查是否有code字段，如果有则按照code判断
                    if (result.data.code !== undefined) {
                        if (result.data.code === 0 && result.data.data && result.data.data.systemWeb
Menu) {
                            try {
                                let config;
                                if (typeof result.data.data.systemWebMenu === 'string') {
                                    // 如果是字符串，需要解析JSON
                                    config = JSON.parse(result.data.data.systemWebMenu);
                                } else {
                                    // 如果已经是对象，直接使用
                                    config = result.data.data.systemWebMenu;
                                }

                                // 检查配置中是否包含features对象
                                if (config && config.features) {
                                    // 确保knowledgeBase功能存在且配置正确
                                    if (!config.features.knowledgeBase) {
                                        console.warn('配置中缺少knowledgeBase功能，合并默认配置');
                                        config.features = { ...this.defaultFeatures, ...config.featu
res };
                                    }
                                }
                                resolve(config.features);
                            } else {
                                console.warn('配置中缺少features对象，使用默认配置');
                                resolve(this.defaultFeatures);
                            }
                            configCache.systemWebMenu = config;
                        } catch (error) {
                            console.warn('处理systemWebMenu配置失败:', error);
                            resolve(null);
                        }
                    }
                } else {
                    console.warn('接口返回code不为0或缺少必要数据，使用默认配置');
                    resolve(null);
                }
            } else {
                // 如果没有code字段，直接检查systemWebMenu
                if (result.data && result.data.systemWebMenu) {
                    try {
                        let config;
                        if (typeof result.data.systemWebMenu === 'string') {
                            // 如果是字符串，需要解析JSON
                            config = JSON.parse(result.data.systemWebMenu);
                        } else {

```

```

        // 如果已经是对象，直接使用
        config = result.data.systemWebMenu;
    }

    // 检查配置中是否包含features对象
    if (config && config.features) {
        // 确保knowledgeBase功能存在且配置正确
        if (!config.features.knowledgeBase) {
            console.warn('配置中缺少knowledgeBase功能，合并默认配置');
            config.features = { ...this.defaultFeatures, ...config.featu
res };

            }
            resolve(config.features);
        } else {
            console.warn('配置中缺少features对象，使用默认配置');
            resolve(this.defaultFeatures);
        }
        configCache.systemWebMenu = config;
    } catch (error) {
        console.warn('处理systemWebMenu配置失败:', error);
        resolve(null);
    }
} else {
    console.warn('接口返回缺少systemWebMenu数据，使用默认配置');
    resolve(null);
}
}
this.updateConfigCache(configCache)
} else {
    console.warn('接口返回数据中缺少data字段，使用默认配置');
    resolve(null);
}
} else {
    console.warn('pub-config接口调用失败，使用默认配置');
    resolve(null);
}
});
});
}

/**
 * 获取当前配置
 */
getCurrentConfig() {
    // 返回内存中的当前配置
    return this.currentFeatures;
}

/**
 * 保存配置到后端API
 */
async saveConfig(config) {
    try {
        // 更新内存中的配置
        this.currentFeatures = { ...config };

        // 异步保存到后端API
        this.saveConfigToAPI(config).catch(error => {
            console.warn('保存配置到API失败:', error);
        }).finally(() => {

```

```

        this.init()
    });

    // 触发配置变更事件
    window.dispatchEvent(new CustomEvent('featureConfigChanged', {
        detail: config
    }));
} catch (error) {
    console.error('保存功能配置失败:', error);
}
}

/**
 * 保存配置到后端API
 */
async saveConfigToAPI(config) {
    return new Promise((resolve) => {
        // 直接使用已知的ID (600) 更新参数
        Api.admin.updateParam(
            {
                id: 600,
                paramCode: 'system-web.menu',
                paramValue: JSON.stringify({
                    features: config,
                    groups: {
                        featureManagement: ["voiceprintRecognition", "voiceClone", "knowledgeBase",
"mcpAccessPoint", "addressBook"],
                        voiceManagement: ["vad", "asr"]
                    }
                }),
                valueType: 'json',
                remark: '系统功能菜单配置'
            },
            (updateResult) => {
                if (updateResult.code === 0) {
                    resolve();
                } else {
                    // 如果更新失败, 可能是参数不存在或其他错误, 记录但不阻止保存到localStorage
                    console.warn('更新参数失败:', updateResult.msg);
                    resolve(); // 不阻止保存到localStorage
                }
            },
            (error) => {
                console.warn('更新参数失败:', error);
                resolve(); // 不阻止保存到localStorage
            }
        );
    });
}

/**
 * 获取所有功能配置
 */
getAllFeatures() {
    return this.getCurrentConfig();
}

/**

```

```
* 获取简化的配置对象（用于首页组件）
*/
getConfig() {
  const features = this.getAllFeatures();
  return {
    voiceprintRecognition: features.voiceprintRecognition?.enabled || false,
    voiceClone: features.voiceClone?.enabled || false,
    knowledgeBase: features.knowledgeBase?.enabled || false,
    mcpAccessPoint: features.mcpAccessPoint?.enabled || false,
    vad: features.vad?.enabled || false,
    asr: features.asr?.enabled || false,
    addressBook: features.addressBook?.enabled || false
  };
}

/**
 * 获取指定功能的状态
 */
getFeatureStatus(featureKey) {
  const features = this.getAllFeatures();
  return features[featureKey]?.enabled || false;
}

/**
 * 设置功能状态
 */
setFeatureStatus(featureKey, enabled) {
  const features = this.getAllFeatures();
  if (features[featureKey]) {
    features[featureKey].enabled = enabled;
    this.saveConfig(features);
    return true;
  }
  return false;
}

/**
 * 启用功能
 */
enableFeature(featureKey) {
  return this.setFeatureStatus(featureKey, true);
}

/**
 * 禁用功能
 */
disableFeature(featureKey) {
  return this.setFeatureStatus(featureKey, false);
}

/**
 * 切换功能状态
 */
toggleFeature(featureKey) {
  const currentStatus = this.getFeatureStatus(featureKey);
  return this.setFeatureStatus(featureKey, !currentStatus);
}

/**
 * 重置所有功能为默认状态
```

```

    */
    resetToDefault() {
        this.saveConfig(this.defaultFeatures);
    }

    /**
     * 批量更新功能状态
     */
    updateFeatures(featureUpdates) {
        const features = this.getAllFeatures();
        Object.keys(featureUpdates).forEach(featureKey => {
            if (features[featureKey]) {
                features[featureKey].enabled = featureUpdates[featureKey];
            } else if (this.defaultFeatures[featureKey]) {
                features[featureKey] = { ...this.defaultFeatures[featureKey] };
                features[featureKey].enabled = featureUpdates[featureKey];
            }
        });
        this.saveConfig(features);
    }

    /**
     * 获取已启用的功能列表
     */
    getEnabledFeatures() {
        const features = this.getAllFeatures();
        return Object.keys(features).filter(key => features[key].enabled);
    }

    /**
     * 检查功能是否启用
     */
    isFeatureEnabled(featureKey) {
        return this.getFeatureStatus(featureKey);
    }
}

// 创建单例实例
const featureManager = new FeatureManager();

export default featureManager;
// ===== 文件: src/utils/format.js =====
// 日期格式化函数
export function formatDate(date) {
    if (!date) return '';
    const d = new Date(date);
    const year = d.getFullYear();
    const month = String(d.getMonth() + 1).padStart(2, '0');
    const day = String(d.getDate()).padStart(2, '0');
    const hour = String(d.getHours()).padStart(2, '0');
    const minute = String(d.getMinutes()).padStart(2, '0');
    const second = String(d.getSeconds()).padStart(2, '0');
    return `${year}-${month}-${day} ${hour}:${minute}:${second}`;
}

// 文件大小格式化函数
export function formatFileSize(bytes) {
    if (!bytes || bytes === 0) return '0 B';
    const units = ['B', 'KB', 'MB', 'GB', 'TB'];
    const k = 1024;

```

```

    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + units[i];
}
// ===== 文件: src/utils/index.js =====
import { Message } from 'element-ui'
import router from '../router'
import Constant from '../utils/constant'

/**
 * 判断用户是否登录
 */
export function checkUserLogin(fn) {
    let token = localStorage.getItem(Constant.STORAGE_KEY.TOKEN)
    let userType = localStorage.getItem(Constant.STORAGE_KEY.USER_TYPE)
    if (isNull(token) || isNull(userType)) {
        goToPage('console', true)
        return
    }
    if (fn) {
        fn()
    }
}

/**
 * 判断是否为空
 * @param data
 * @returns {boolean}
 */
export function isNull(data) {
    if (data === undefined) {
        return true
    } else if (data === null) {
        return true
    } else if (typeof data === 'string' && (data.length === 0 || data === '' || data === 'undefined' || data === 'null')) {
        return true
    } else if ((data instanceof Array) && data.length === 0) {
        return true
    }
    return false
}

/**
 * 判断不为空
 * @param data
 * @returns {boolean}
 */
export function isNotNull(data) {
    return !isNull(data)
}

/**
 * 显示顶部红色通知
 * @param msg
 */
export function showDanger(msg) {
    if (isNull(msg)) {
        return
    }
    Message({

```

```
        message: msg,
        type: 'error',
        showClose: true
    })
}

/**
 * 显示顶部橙色通知
 * @param msg
 */
export function showWarning(msg) {
    if (isNull(msg)) {
        return
    }
    Message({
        message: msg,
        type: 'warning',
        showClose: true
    });
}

/**
 * 显示顶部绿色通知
 * @param msg
 */
export function showSuccess(msg) {
    Message({
        message: msg,
        type: 'success',
        showClose: true
    })
}

/**
 * 页面跳转
 * @param path
 * @param isRepalce
 */
export function goToPage(path, isRepalce) {
    if (isRepalce) {
        router.replace(path)
    } else {
        router.push(path)
    }
}

/**
 * 获取当前vue页面名称
 * @param path
 * @param isRepalce
 */
export function getCurrentPage() {
    let hash = location.hash.replace('#', '')
    if (hash.indexOf('?') > 0) {
        hash = hash.substring(0, hash.indexOf('?'))
    }
}
```



```
    return hash
  }

/**
 * 生成从[min,max]的随机数
 * @param min
 * @param max
 * @returns {number}
 */
export function randomNum(min, max) {
  return Math.round(Math.random() * (max - min) + min)
}

/**
 * 获取uuid
 */
export function getUUID() {
  return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, c => {
    return (c === 'x' ? (Math.random() * 16 | 0) : ('r&0x3' | '0x8')).toString(16)
  })
}

/**
 * 验证手机号格式
 * @param {string} mobile 手机号
 * @param {string} areaCode 区号
 * @returns {boolean}
 */
export function validateMobile(mobile, areaCode) {
  // 移除所有非数字字符
  const cleanMobile = mobile.replace(/\D/g, '');

  // 根据不同区号使用不同的验证规则
  switch (areaCode) {
    case '+86': // 中国大陆
      return /^[13-9]\d{9}$/.test(cleanMobile);
    case '+852': // 中国香港
      return /^[569]\d{7}$/.test(cleanMobile);
    case '+853': // 中国澳门
      return /^[6]\d{7}$/.test(cleanMobile);
    case '+886': // 中国台湾
      return /^[9]\d{8}$/.test(cleanMobile);
    case '+1': // 美国/加拿大
      return /^[2-9]\d{9}$/.test(cleanMobile);
    case '+44': // 英国
      return /^[71-9]\d{8}$/.test(cleanMobile);
    case '+81': // 日本
      return /^[7890]\d{8}$/.test(cleanMobile);
    case '+82': // 韩国
      return /^[10-9]\d{7}$/.test(cleanMobile);
    case '+65': // 新加坡
      return /^[89]\d{7}$/.test(cleanMobile);
    case '+61': // 澳大利亚
      return /^[4578]\d{8}$/.test(cleanMobile);
    case '+49': // 德国
      return /^[15-7]\d{8}$/.test(cleanMobile);
    case '+33': // 法国
      return /^[67]\d{8}$/.test(cleanMobile);
```

```

    case '+39': // 意大利
        return /^3[0-9]\d{8}$/.test(cleanMobile);
    case '+34': // 西班牙
        return /^[6-9]\d{8}$/.test(cleanMobile);
    case '+55': // 巴西
        return /^[1-9]\d{10}$/.test(cleanMobile);
    case '+91': // 印度
        return /^[6-9]\d{9}$/.test(cleanMobile);
    case '+971': // 阿联酋
        return /^[5]\d{8}$/.test(cleanMobile);
    case '+966': // 沙特阿拉伯
        return /^[5]\d{8}$/.test(cleanMobile);
    case '+880': // 孟加拉国
        return /^[1[3-9]\d{8}$/.test(cleanMobile);
    case '+234': // 尼日利亚
        return /^[789]\d{9}$/.test(cleanMobile);
    case '+254': // 肯尼亚
        return /^[17]\d{8}$/.test(cleanMobile);
    case '+255': // 坦桑尼亚
        return /^[67]\d{8}$/.test(cleanMobile);
    case '+7': // 哈萨克斯坦
        return /^[67]\d{9}$/.test(cleanMobile);
    default:
        // 其他国际号码: 至少5位, 最多15位
        return /^d{5,15}$/.test(cleanMobile);
}
}

/**
 * 生成SM2密钥对 (十六进制格式)
 * @returns {Object} 包含公钥和私钥的对象
 */
export function generateSm2KeyPairHex() {
    // 使用sm-crypto库生成SM2密钥对
    const sm2 = require('sm-crypto').sm2;
    const keypair = sm2.generateKeyPairHex();

    return {
        publicKey: keypair.publicKey,
        privateKey: keypair.privateKey,
        clientPublicKey: keypair.publicKey, // 客户端公钥
        clientPrivateKey: keypair.privateKey // 客户端私钥
    };
}

/**
 * SM2公钥加密
 * @param {string} publicKey 公钥 (十六进制格式)
 * @param {string} plainText 明文
 * @returns {string} 加密后的密文 (十六进制格式)
 */
export function sm2Encrypt(publicKey, plainText) {
    if (!publicKey) {
        throw new Error('公钥不能为null或undefined');
    }

    if (!plainText) {
        throw new Error('明文不能为空');
    }
}

```

```

const sm2 = require('sm-crypto').sm2;
// SM2加密, 添加04前缀表示未压缩公钥
const encrypted = sm2.doEncrypt(plainText, publicKey, 1);
// 转换为十六进制格式 (与后端保持一致, 添加04前缀)
const result = "04" + encrypted;

return result;
}

/**
 * SM2私钥解密
 * @param {string} privateKey 私钥 (十六进制格式)
 * @param {string} cipherText 密文 (十六进制格式)
 * @returns {string} 解密后的明文
 */
export function sm2Decrypt(privateKey, cipherText) {
  const sm2 = require('sm-crypto').sm2;
  // 移除04前缀 (与后端保持一致)
  const dataWithoutPrefix = cipherText.startsWith("04") ? cipherText.substring(2) : cipherText;
  // SM2解密
  return sm2.doDecrypt(dataWithoutPrefix, privateKey, 1);
}

/**
 * 防抖函数
 * @param {Function} fn 要防抖的函数
 * @param {number} delay 延迟时间 (毫秒), 默认500ms
 * @param {boolean} immediate 是否立即执行, 默认false
 * @returns {Function} 防抖处理后的函数
 */
export function debounce(fn, delay = 500, immediate = false) {
  let timer = null;

  return function (...args) {
    const context = this;

    if (timer) {
      clearTimeout(timer);
    }

    if (immediate && !timer) {
      fn.apply(context, args);
    }

    timer = setTimeout(() => {
      if (!immediate) {
        fn.apply(context, args);
      }
      timer = null;
    }, delay);
  };
}

// ===== 文件: src/i18n/index.js =====
import Vue from 'vue';
import VueI18n from 'vue-i18n';
import zhCN from './zh_CN';
import zhTW from './zh_TW';
import en from './en';

```

```
import de from './de';
import vi from './vi';
import ptBR from './pt_BR';

import enLocale from 'element-ui/lib/locale/lang/en'
import zhLocale from 'element-ui/lib/locale/lang/zh-CN'
import twLocale from 'element-ui/lib/locale/lang/zh-TW'
import deLocale from 'element-ui/lib/locale/lang/de'
import viLocale from 'element-ui/lib/locale/lang/vi'
import ptBRLocale from 'element-ui/lib/locale/lang/pt-br'

Vue.use(VueI18n);

// 从本地存储获取语言设置，如果没有则使用浏览器语言或默认语言
// 目前仅开放中文简体和繁体，其他语言（en/de/vi/pt_BR）暂时屏蔽，统一回退到 zh_CN
const getDefaultLanguage = () => {
  const savedLang = localStorage.getItem('userLanguage');
  if (savedLang === 'zh_CN' || savedLang === 'zh_TW') {
    return savedLang;
  }
  const browserLang = navigator.language || navigator.userLanguage;
  if (browserLang === 'zh-TW' || browserLang === 'zh-HK' || browserLang === 'zh-MO') {
    return 'zh_TW';
  }
  // if (browserLang.indexOf('de') === 0) {
  //   return 'de';
  // }
  // if (browserLang.indexOf('vi') === 0) {
  //   return 'vi';
  // }
  // if (browserLang === 'pt-BR' || browserLang === 'pt') {
  //   return 'pt_BR';
  // }
  // return 'en';
  return 'zh_CN';
};

const i18n = new VueI18n({
  locale: getDefaultLanguage(),
  fallbackLocale: 'zh_CN',
  messages: {
    'zh_CN': { ...zhLocale, ...zhCN },
    'zh_TW': { ...twLocale, ...zhTW },
    'en': { ...en, ...enLocale },
    'de': { ...de, ...deLocale },
    'vi': { ...vi, ...viLocale },
    'pt_BR': { ...ptBR, ...ptBRLocale }
  }
});

export default i18n;

// 提供一个方法来切换语言
export const changeLanguage = (lang) => {
  i18n.locale = lang;
  localStorage.setItem('userLanguage', lang);
  // 通知组件语言已更改
  Vue.prototype.$eventBus.$emit('languageChanged', lang);
};
```